

X-Watch 模块架构图册

调研 8 个开源手表的硬件选型和固件组织，把验证过的做法落到 X-Watch：模块怎么分、怎么通信、怎么省电、怎么升级。每条都注明出处，没在源码或文档里核对过的标了 **推断**。

工程	X-Watch
硬件版本	Rev1.0
平台	STM32L4R9 · TouchGFX
配套文档	系统架构图册
编写日期	2026-09-17

调研对象：8 个开源手表各有什么长处

入选标准：软硬件公开、有出货或活跃社区。覆盖彩屏、墨水屏、反射式 LCD、段码屏，和 X-Watch 最接近的是 PineTime、ZSWatch、OV-Watch。

项目	主控 · 内存	屏幕	系统 · 图形	最值得学的地方
PineTime / InfiniTime	nRF52832 · 64 KB RAM · 512 KB Flash + 4 MB SPI NOR	1.3" 240×240 IPS · ST7789 · CST816S 触摸	FreeRTOS · LVGL · C++	控制器 (Controller) 与应用解耦；MCUBoot 双槽升级 + 固件确认 + 恢复固件
ZSWatch	nRF5340 · 512 KB RAM · 1 MB Flash + 64 MB 外部 Flash	1.28" 240×240 圆形 IPS · 触摸	Zephyr · LVGL · C	zbus 发布订阅事件总线；三态电源管理；iOS 用 ANCS/AMS/CTS，安卓用 Gadgetbridge
PebbleOS	多代硬件；新款 nRF52840	墨水屏 / 反射式 LCD	FreeRTOS · 自研 · C	applib / services / kernel / drivers 严格分层；第三方应用非特权运行，经系统调用访问内核；自研 PFS 文件系统
Bangle.js 2	nRF52840 · 256 KB RAM · 1 MB Flash + 8 MB 外部 Flash	1.3" 176×176 反射式 LCD · 常亮	Espruino · JavaScript	常亮低功耗屏：175 mAh 电池待机 4 周；应用可通过浏览器无线上传
OV-Watch	STM32F411CEU6	彩色触摸屏	FreeRTOS · LVGL 8.2	同为 STM32 方案；页面栈导航；HWDatAccess 中间层让同一套界面代码在 PC 模拟器和硬件上都能跑；Boot 与 App 分离后支持蓝牙 OTA
Watchy	ESP32	1.54" 200×200 墨水屏	Arduino	MCU 平时深度睡眠，外部 RTC 每分钟唤醒一次刷新；只走时可用 5 ~ 7 天
Sensor Watch	SAM L22 · 32 KB RAM · 256 KB Flash	卡西欧 F-91W 段码 LCD	Movement 框架 · C	MCU 每秒醒一次、只醒极短时间，整机低于 30 μA；表盘即插件，社区贡献了几十个
Open-SmartWatch	ESP32	圆形彩屏	Arduino · 自研	硬件模块化，可自行组装和维修；OswAppV2 统一应用基类；带模拟器与单元 / 界面测试

共同点 三个彩屏方案都是"RTOS + 图形库 + 应用框架"，都用 IMU 做抬腕亮屏；PineTime、ZSWatch、Bangle.js 都外挂 Flash 存资源或升级包。

X-Watch 的差异 主控无射频，蓝牙外挂；390×390 AMOLED 分辨率更高；用 TouchGFX 而非 LVGL。 **许可证** OV-Watch 为 GPL-3.0，PebbleOS 为 Apache-2.0，只借鉴思路，复制代码前先核对。

长处清单：学什么、从哪学、怎么落到 X-Watch

下表按固件组织、功耗、升级与连接三类分组。"采纳"是 Rev1.0 就要实现的，"预留"是先在架构上留好接口、后续版本再做的。

长处	来源	原项目怎么做	X-Watch 怎么落地	决策
固件组织				
事件总线解耦	ZSWatch	传感器、蓝牙、管理器、应用之间只通过命名通道收发消息，互不直接调用	基于 FreeRTOS 队列实现轻量发布订阅，见第 5 页	采纳
控制器持有资源	InfiniTime	components/ 下 battery、ble、datetime、motion、heartrate 等控制器，应用只引用控制器	services 层每个资源一个服务对象，TouchGFX 的 Model 只和服务打交道	采纳
严格分层	PebbleOS	applib 应用框架 / services / kernel / drivers，另有 shell 管启动器和表盘流程	在现有 app · services · drivers · hal 四层上增加 shell 与 managers	采纳
应用编译期注册	InfiniTime	Apps 枚举 + AppTraits，CMake 用 ENABLE_USERAPPS 选择编译哪些应用	应用描述表在编译期生成，按产品配置裁剪应用，见第 7 页	采纳
硬件访问中间层	OV-Watch · OSW	一个开关在"模拟数据"与"真实硬件"之间切换；OSW 另有单元测试与界面测试	services 接口下挂真实实现和模拟实现两套，TouchGFX 模拟器直接链接模拟实现	采纳
应用沙箱	PebbleOS	第三方应用非特权运行，通过 sys_* 系统调用进入内核，MPU 隔离	Rev1.0 只有自研应用，不做沙箱；MCU 有 MPU，可以后续再加	预留
功耗				
三态电源管理	ZSWatch	活跃 / 非活跃 / 未佩戴静止三种状态，状态变化广播给应用	亮屏 / 息屏 / 未佩戴 / 关机四态，见第 8 页	采纳
定时器按需启停	ZSWatch	100 ms / 1 s / 10 s 周期通道，没有订阅者时定时器自动停止	秒 tick 只有在有订阅者时运行，息屏后表盘退订	采纳
短暂唤醒	Sensor Watch · Watchy	MCU 大部分时间睡眠，每秒或每分钟醒来极短时间干完活再睡	息屏时 RTC 按分钟唤醒做计步汇总，其余交给 IMU 内部处理	采纳
升级与连接				
双槽升级 + 确认 + 回滚	InfiniTime	新固件先写外部 Flash，Bootloader 交换后运行；用户不确认就回滚到旧版	OctoSPI 上设 OTA 暂存槽，Bootloader 负责校验、交换与回滚，见第 9 页	采纳
恢复固件	InfiniTime	精简版固件只保留蓝牙和 OTA，主固件坏了也能救回来	在外部 Flash 放一份恢复固件，长按按键进入	采纳
标准蓝牙协议	ZSWatch · InfiniTime	iOS 走 ANCS / AMS / CTS；安卓兼容 Gadgetbridge	不自研手机 App 也能收通知、同步时间，见第 10 页	采纳
浏览器装应用	Bangle.js	应用是脚本，通过网页经蓝牙上传	TouchGFX 是编译型框架，不适用；只保留"表盘资源包"式的轻量替换	不采纳

硬件模块：按功能切成 8 块，块与块之间只留约定好的接口

借鉴 Open-SmartWatch 和 ZSWatch：硬件按功能分块，每块有明确的接口和电源边界。这样后续版本可以单独替换某一块，比如换屏或换蓝牙方案，不牵动其他块。Rev1.0 仍是一整块主板，模块划分体现在原理图分页和 boards/board_config.h 的引脚分组上。

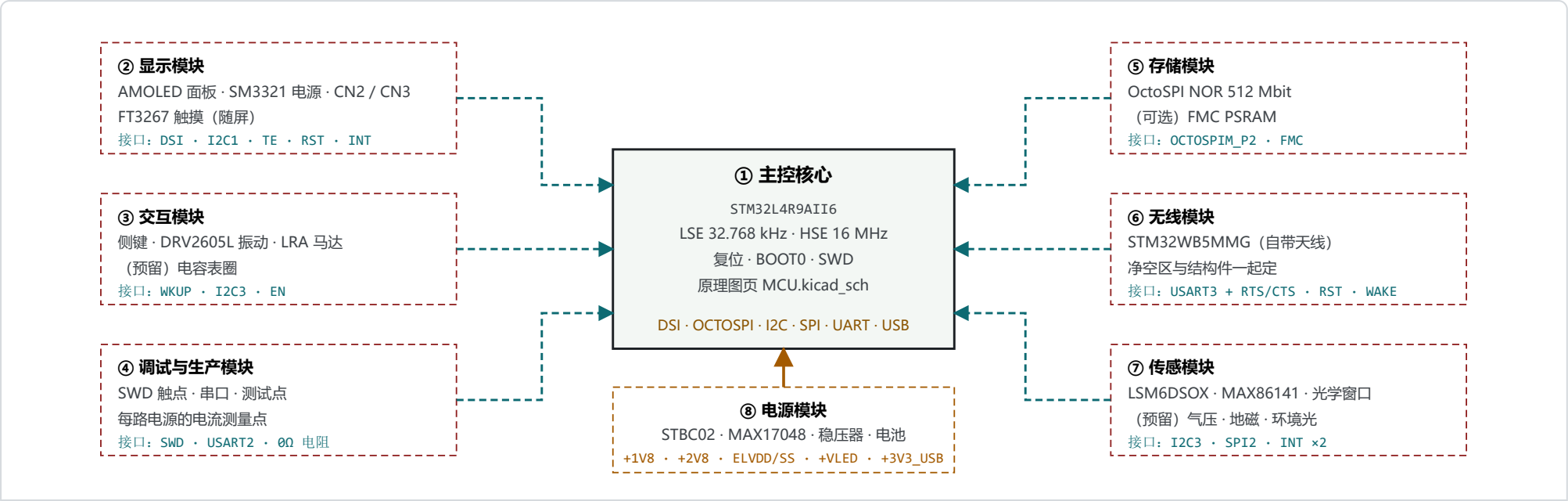


图 1 硬件模块划分。虚线框是模块边界，青色虚线是模块对主控的接口，电源模块给所有模块供电（图中只画出到主控的一路）。每个模块对应一张原理图子页，子页名与模块名一致，引脚在 board_config.h 里按模块分组。

边界约定	做法	借鉴
电源可独立关断	显示、无线、传感三个模块各自有负载开关或使能脚，息屏和未佩戴时整块断电	ZSWatch 按状态关外设；PineTime 息屏关背光 推断
中断统一上报	每个模块最多占用 1 ~ 2 根中断线，统一接 EXTI；不在模块间直连中断	ZSWatch 的 IMU 手势走中断而不是轮询
测试点齐全	每个模块的供电、复位、通信线都留测试点，便于单独调试和生产测试	ZSWatch 仓库带生产测试工具

固件模块总图：六层加一条事件总线

在仓库现有的 app · services · drivers · hal 四层之上，参照 PebbleOS 增加 shell（启动器与导航），参照 ZSWatch 增加 managers（全局状态）和事件总线。规则只有一条：下层不知道上层存在，同层之间只通过事件总线通信。

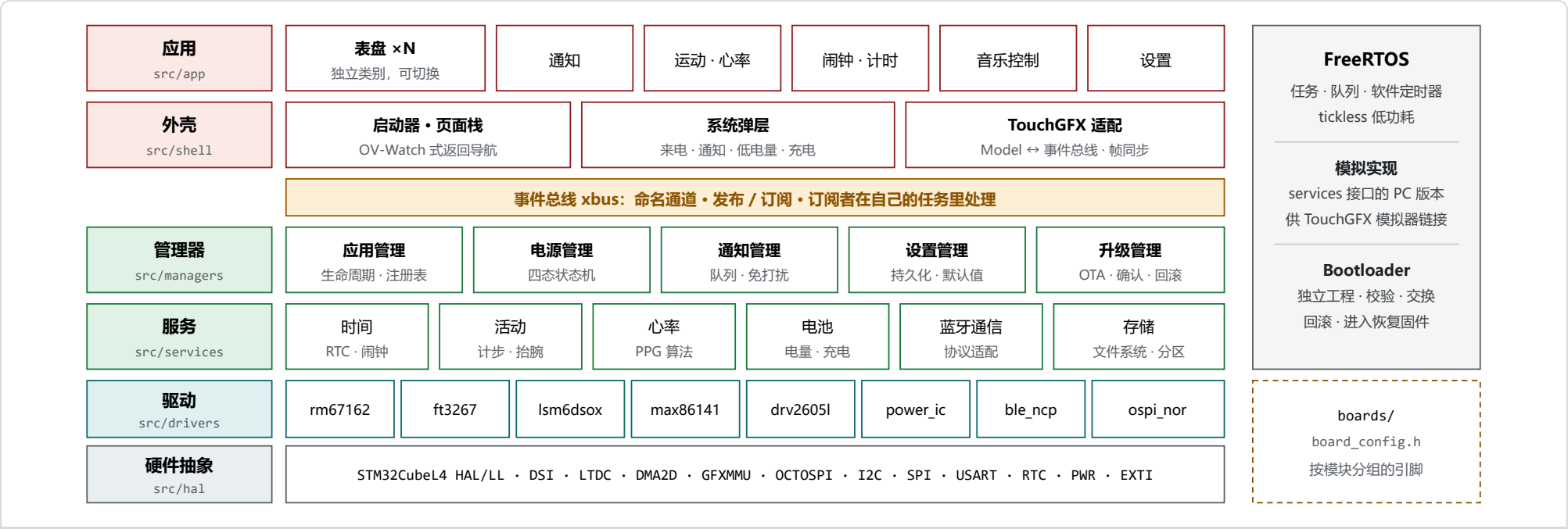


图 2 固件模块。红色是界面相关（只在 GUI 任务里运行），绿色是业务逻辑，青色是器件驱动，灰色是芯片外设。事件总线位于界面和业务之间：应用只订阅通道，不直接调用服务；服务只发布数据，不知道谁在用。模块名是建议值，xbus 指 X-Watch 自己实现的总线。

层	职责边界	借鉴
外壳 shell	决定"现在显示哪个应用": 启动器、页面栈、表盘切换、系统弹层; 应用本身不能跳到别的应用, 只能请求外壳	PebbleOS src/fw/shell; OV-Watch 页面栈
管理器 managers	跨应用的全局状态, 只有一份实例: 当前电源状态、通知队列、设置、升级进度	ZSWatch Managers 层; InfiniTime NotificationManager
服务 services	每个硬件资源一个服务, 负责采样、算法、缓存, 并把结果发布到总线; 接口有真实与模拟两种实现	InfiniTime Controllers; OV-Watch HWDDataAccess

事件总线：通道清单与使用规则

ZSWatch 用 Zephyr 自带的 zbus，X-Watch 跑 FreeRTOS，没有现成的总线，需要自己实现一个轻量版：通道是静态定义的结构体，保存最后一条消息和订阅者列表；发布时把消息拷进每个订阅者任务的队列。ZSWatch 文档特别提醒，监听回调运行在发布者的上下文里，重活要转交工作队列，X-Watch 直接规定只走队列。

通道	消息内容	发布者	典型订阅者	频率
时间与状态				
tick_1s_chan · tick_1min_chan	秒 / 分钟节拍	时间服务	表盘、秒表、活动服务	按需启停：没有订阅者就不跑
power_state_chan	亮屏 / 息屏 / 未佩戴 / 关机	电源管理	应用管理、所有服务、TouchGFX 适配	状态变化时
input_chan	按键短按 / 长按、触摸手势	交互驱动	外壳、电源管理（重置息屏计时）	事件
传感与电池				
motion_chan	抬腕、放下、双击、步数增量、是否佩戴	活动服务（IMU 中断）	电源管理、运动应用	事件 + 每分钟汇总
heart_rate_chan	心率值、置信度、测量状态	心率服务	运动应用、蓝牙通信	测量期间 1 s
battery_chan	电量百分比、电压、充电状态	电池服务	表盘、外壳（低电量弹层）、电源管理	1 min 或状态变化
手机通信				
ble_link_chan	连接、断开、配对状态	蓝牙通信	表盘状态图标、通知管理	事件
phone_msg_chan	通知、来电、时间同步、天气、音乐信息	蓝牙通信	通知管理、时间服务、音乐应用	事件
phone_cmd_chan	音乐控制、查找手机、拒接来电	应用	蓝牙通信	事件
ota_chan	升级开始、进度、完成、失败	升级管理	外壳（进度页）、电源管理（升级期间禁止休眠）	进度变化时

规则 1 · 只拷贝不共享 必须

消息按值拷贝进队列，大小固定，不传指针。通知正文这类长数据放通知管理的缓冲区，消息里只带编号。

规则 2 · 可读最后一条 建议

通道保存最新一条消息。应用切到前台时直接读取当前电量、心率，不用等下一次发布，界面不会出现空白。

规则 3 · GUI 任务限流 推断

TouchGFX 每帧处理一次队列，高频通道（心率、秒节拍）在适配层合并，只保留最新值，避免队列积压拖慢刷新。

任务模型：几个任务、谁负责什么

InfiniTime 在单核 nRF52832 上只跑少数几个任务：SystemTask（初始化、事件路由、休眠唤醒）、DisplayApp（运行应用、控制屏幕、处理触摸），外加蓝牙协议栈的两个任务和心率测量任务。PebbleOS 也是固定任务集：KernelMain、KernelBackground、App、Worker、蓝牙、定时器。X-Watch 沿用"固定少量任务"的思路，蓝牙协议栈在 STM32WB5MMG 里运行，主控不用跑。

任务	职责	唤醒来源	相对优先级	对应开源项目
sys	启动时初始化驱动与服务；运行电源管理与应用管理；处理按键、充电插拔等系统事件	input_chan、power_state_chan、看门狗节拍	高	InfiniTime SystemTask；PebbleOS KernelMain
gui	运行 TouchGFX 主循环；外壳与所有应用界面都在这里；把总线消息交给 Model	TE 帧同步、触摸中断、订阅的通道	中	InfiniTime DisplayApp；PebbleOS App
sensor	活动、心率、电池三个服务；读 IMU FIFO 与 PPG FIFO，跑算法，发布结果	IMU / PPG 中断、分钟节拍	中	InfiniTime HeartRateTask；ZSWatch 传感器数据管线
comm	蓝牙通信服务：与 WB5MMG 的 UART 协议收发、协议适配（ANCS / Gadgetbridge）、OTA 数据写入外部 Flash	UART 接收中断、phone_cmd_chan	中高	InfiniTime 的 ble 任务；PebbleOS comm
bg	后台慢活：文件系统写入、设置保存、日志、运动记录落盘	工作队列	低	PebbleOS KernelBackground
定时器服务	FreeRTOS 软件定时器：息屏计时、长按判定、按需启停的秒节拍	—	系统默认	ZSWatch 周期通道的按需启停

为什么不给每个服务开任务

说明

- 每个任务都要独立的栈，640 KB SRAM 里帧缓冲已经占掉大头（见系统架构图册第 3 页）。
- 任务越多，进入 Stop 2 前要确认"全部空闲"的对象越多。把同类低频服务合进一个 sensor 任务，更容易做到长时间睡眠 **推断**。

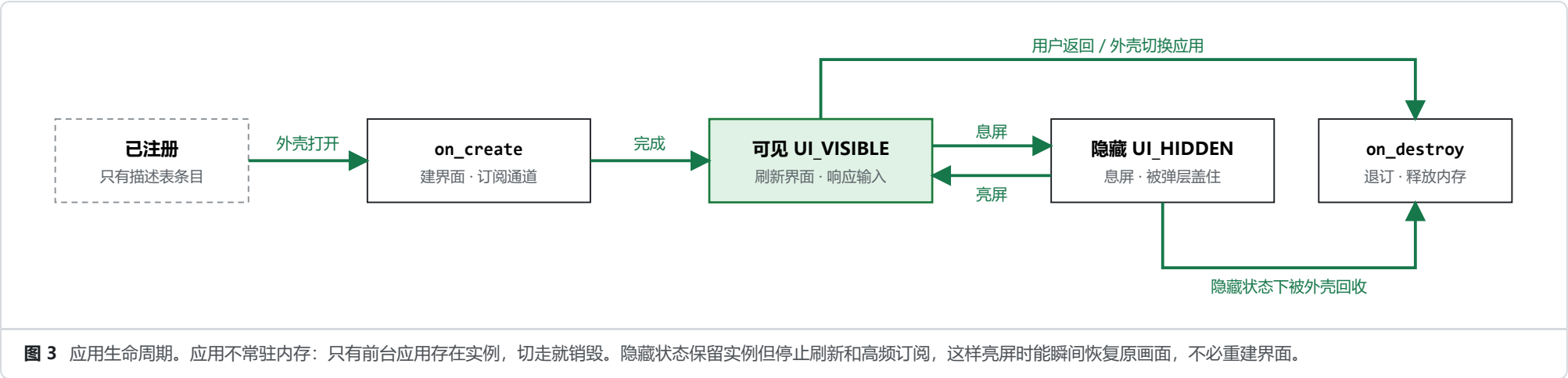
唤醒与休眠的判断放在 sys

建议

- InfiniTime 把"管理唤醒和睡眠模式"明确交给 SystemTask，其他任务只报告自己是否忙。
- X-Watch 同样由 sys 统一决定何时关屏、何时允许 tickless 空闲进入 Stop 2；gui 任务在息屏状态下阻塞在队列上，不占 CPU。

应用框架：注册、生命周期与 TouchGFX 映射

InfiniTime 的应用继承同一个 Screen 基类，打开时创建、退出时销毁，通过 AppTraits 在编译期登记，CMake 选择编译哪些应用，表盘单独一类。ZSWatch 在应用不可见时发出 UI_HIDDEN，让应用暂停界面工作。X-Watch 把这两点合起来，再映射到 TouchGFX 的 Screen / Presenter / Model 结构上。



框架要素	X-Watch 做法
应用描述表	每个应用一条：编号、名称、图标、类别（表盘 / 应用 / 系统）、创建函数、所需通道；编译期生成
按产品裁剪	CMake 选项列出要编译的应用和表盘，与 InfiniTime 的 ENABLE_USERAPPS / ENABLE_WATCHFACES 同理
导航	外壳维护页面栈：打开应用时入栈，返回时出栈，侧键长按直接回表盘
输入分发	外壳先处理全局手势（下滑通知、侧键），剩余事件交给前台应用

TouchGFX 结构	在 X-Watch 里承担什么
View (Screen)	纯界面：控件布局和动画，不引用任何服务
Presenter	应用逻辑：处理输入、决定显示什么；对应生命周期的创建与销毁
Model	全局唯一，是界面与事件总线之间的桥：每帧取出 gui 队列里的消息，通知当前 Presenter
FrontendApplication	外壳的落点：屏幕切换、页面栈、系统弹层

电源管理：四种状态与切换条件

ZSWatch 把电源分为活跃、非活跃、未佩戴静止三态，并广播给应用。Sensor Watch 和 Watchy 证明了"大部分时间睡、短暂醒来干活"的威力：前者整机低于 30 μ A，后者只走时可用 5 ~ 7 天。OV-Watch 同为 STM32 方案，实测运行 70 ~ 80 mA、STOP 睡眠约 1 mA，说明睡眠电流主要由外设决定 **推断**。X-Watch 取四态，每个状态明确关掉哪些模块。



图 4 电源状态机，由 sys 任务里的电源管理维护，每次切换都发布到 power_state_chan。"是否佩戴"可以由心率 AFE 的接近检测或 IMU 长时间静止来判断 **推断**；具体用哪种要看样板实测的误判率。

模块	亮屏	息屏	未佩戴	关机
主控	Run，界面刷新时满速	Stop 2，RTC 与 EXTI 可唤醒	Stop 2	Shutdown，只留 WKUP 引脚
显示	点亮，命令模式按需刷新	面板断电，或进待机以便快速亮屏 推断	断电	断电
传感	IMU 正常；心率按应用需求	IMU 低功耗 + 抬腕中断；心率周期测	IMU 只保留运动唤醒；心率关	断电
无线	保持连接，短连接间隔	保持连接，长连接间隔	更长间隔或只广播	断电
交互	振动可用	振动驱动关断，收到通知时临时打开	关断	电源键仍可唤醒

存储与升级：Bootloader、双槽、恢复固件

InfiniTime 的升级链路最成熟：新固件先写进外部 SPI Flash 的第二槽，Bootloader (MCUBoot) 检查到新固件就与主槽交换再运行；用户在设置里确认之前，重启会回滚到旧版；主固件彻底坏掉时，长按按键可以装回一份只有蓝牙和 OTA 功能的恢复固件。OV-Watch 也在 2.4 版把 Boot 与 App 分离，实现了蓝牙 OTA。X-Watch 照此设计。



图 5 存储分区。资源和升级槽都在外部 Flash，这一点与 InfiniTime、ZSWatch、Bangle.js 一致。资源分区的版本必须和固件匹配，升级包要能同时携带固件和资源。

环节	X-Watch 做法	借鉴	决策
下载	comm 任务把 OTA 数据写入第二槽，写完校验整包哈希；期间 ota_chan 广播进度，电源管理禁止休眠	InfiniTime DfuService	采纳
交换	Bootloader 检测第二槽有新固件：校验通过才与主槽交换，旧固件留在第二槽	MCUBoot 交换机制	采纳
确认	新固件启动后进入“待确认”：自检（屏、触摸、蓝牙）通过或用户确认才写确认标志，否则下次重启回滚	InfiniTime 固件确认；components/firmwarevalidator	采纳
救砖	开机时长按侧键进入 Bootloader 的恢复选项，从外部 Flash 装回恢复固件	InfiniTime 恢复固件	采纳
双 bank	MCU 片内 Flash 支持双 bank，可以不经外部 Flash 直接在片内 A/B 切换；但每个 bank 只有 1 MB，固件可能放不下 推断	STM32L4R9 规格书	备选
文件系统	设置与运动记录用带磨损均衡、掉电安全的文件系统（LittleFS 或同类）；PebbleOS 的 PFS 用轮转分配页面实现磨损均衡	InfiniTime components/fs；PebbleOS PFS	采纳

手机通信：不写 App 也能用的协议组合

ZSWatch 的做法很务实：iOS 直接用苹果开放的 ANCS（通知）、AMS（音乐）、CTS（时间）标准服务；安卓兼容开源 App Gadgetbridge 的协议。InfiniTime 则实现了一整套 GATT 服务，包括告警通知、电池、当前时间、设备信息、DFU、文件、心率、运动、音乐、导航、天气。X-Watch 第一版不自研手机 App，靠这些现成协议覆盖核心功能。

功能	iOS	安卓	手表侧模块	参考实现
消息通知	ANCS（手表作为客户端订阅）	Gadgetbridge 协议	蓝牙通信 → phone_msg_chan → 通知管理	ZSWatch；InfiniTime AlertNotificationService / Client
时间同步	CTS 当前时间服务	Gadgetbridge 协议	蓝牙通信 → 时间服务写 RTC	InfiniTime CurrentTimeService / Client
音乐控制	AMS	Gadgetbridge 协议	音乐应用 ↔ phone_cmd_chan	ZSWatch；InfiniTime MusicService
天气·导航	需要自研 App 或快捷指令	Gadgetbridge 协议	蓝牙通信 → phone_msg_chan	InfiniTime SimpleWeatherService / NavigationService
健康数据	标准心率服务（HRS）	标准心率服务	心率服务 → 蓝牙通信	InfiniTime HeartRateService
固件升级	自定义 DFU 服务；Gadgetbridge 已支持给 InfiniTime 升级，协议兼容可直接复用 推断		升级管理 + comm 任务	InfiniTime DfuService
电量·设备信息	标准电池服务（BAS）与设备信息服务（DIS）		电池服务 → 蓝牙通信	InfiniTime BatteryInformation / DeviceInformation

主控与蓝牙模块怎么分工 推断

- STM32WB5MMG 内部跑 BLE 协议栈和 GATT 服务表，主控通过 UART 用一套简单的帧协议收发"业务消息"。
- 这样主控侧的 comm 任务只关心消息内容，更换蓝牙方案时只需改 WB 侧固件和 ble_ncp 驱动，services 以上不动。

协议适配层单独成模块 建议

- ANCS、Gadgetbridge 等不同来源的通知格式不一样，先在蓝牙通信服务里统一转换成 X-Watch 自己的消息结构，再发布到总线。
- 通知管理和应用只认内部格式，以后接入新的手机协议不影响界面代码。

开发与测试：模拟器、目录结构、生产测试

OV-Watch 用一个开关在"模拟数据"和"真实硬件"之间切换，界面在电脑上就能调；Open-SmartWatch 带模拟器以及单元测试、界面测试；PebbleOS 支持 QEMU 仿真；ZSWatch 仓库里有生产测试工具。TouchGFX Designer 自带 PC 模拟器，X-Watch 只要让 services 有模拟实现，界面开发就能脱离硬件。

目录	内容
firmware/src/app/	表盘与应用，每个应用一个子目录
firmware/src/shell/	启动器、页面栈、系统弹层、TouchGFX 适配
firmware/src/managers/	应用、电源、通知、设置、升级管理
firmware/src/services/	时间、活动、心率、电池、蓝牙通信、存储； 每个服务含 *_hw.c 与 *_sim.c 两种实现
firmware/src/core/	事件总线 xbus、任务定义、日志、断言
firmware/src/drivers/	每个器件一个目录，只依赖 hal
firmware/src/hal/	CubeL4 生成代码与板级初始化
firmware/bootloader/	独立 Bootloader 工程
firmware/gui/	TouchGFX Designer 工程与生成代码
firmware/tests/	在 PC 上运行的单元测试：总线、状态机、算法

测试层级	运行环境	测什么
单元测试	PC	事件总线、电源状态机、计步与心率算法、通知解析
界面测试	TouchGFX 模拟器 + 模拟服务	页面跳转、弹层、各种电量与连接状态下的显示
硬件在环	DISCO 开发板 → X-Watch 样板	屏幕时序、触摸、传感器驱动、各状态电流
生产测试	专用测试固件 + 治具	每个模块自检：屏幕色块、触摸点、IMU、PPG、振动、蓝牙射频、充电

先在开发板上跑通 建议

- 32L4R9IDISCOVERY 带同款屏、外部 Flash 与 PSRAM，事件总线、应用框架、电源状态机都能先在它上面实现。
- X-Watch 样板回来后，只替换 boards/ 与 drivers，上层代码不动。这正是模块化要换来的好处。

实施顺序与待决事项

按依赖关系排序：先搭底座，再接业务，最后做升级与生产。前三步都可以在 DISCO 开发板上完成，不必等 X-Watch 样板。

第 1 步	工程骨架。按第 11 页调整 firmware/ 目录；接入 FreeRTOS 与 TouchGFX（从 STM32L4R9I-DISCO 板级模板起步）；定义五个任务。
第 2 步	事件总线与模拟服务。实现 xbus 和第 5 页的通道；每个服务先写模拟实现，让表盘在 TouchGFX 模拟器里跑起来。
第 3 步	外壳与应用框架。应用描述表、生命周期、页面栈、系统弹层；做出表盘、通知、设置三个应用验证框架。
第 4 步	电源状态机。先在开发板上实现亮屏 / 息屏切换与 Stop 2 进出，测量每个状态的电流，建立基线。
第 5 步	真实服务与驱动。样板回来后逐个把模拟实现换成真实驱动：屏、触摸、IMU、电池、振动、心率。
第 6 步	蓝牙通信。WB5MMG 侧固件与 UART 帧协议；先接 ANCS 与 CTS (iOS)，再接 Gadgetbridge (安卓)。
第 7 步	升级与生产。Bootloader、OTA 第二槽、固件确认与恢复固件；生产测试固件。
待决	事件总线自研还是选现成库。本文按自研轻量实现设计；也可以评估现成的 FreeRTOS 发布订阅组件，比较代码量与内存占用后再定。
待决	佩戴检测方案。心率 AFE 接近检测还是 IMU 静止判断，影响"未佩戴"状态的误判率和传感器常开功耗，需要样板实测。
待决	Bootloader 选型。自研，还是移植 MCUBoot (InfiniTime 的选择)；后者成熟但要适配 OctoSPI 外部 Flash 驱动。

项目资料 InfiniTime 源码与 doc/code (Intro.md、Apps.md、components/) 及 pinetime-mcuboot-bootloader; PINE64 PineTime 规格页; ZSWatch 文档 zswatch.dev 架构页与 GitHub README; PebbleOS 架构文档 (pebbleos-core.readthedocs.io) 与 Google 开源博客; Espruino Bangle.js 2 技术页; OV-Watch GitHub README; Watchy 电池续航文档; Sensor Watch 的 Crowd Supply 页面; Open-SmartWatch 仓库与相关介绍。

标为"推断"的内容 GUI 任务限流方式、任务合并对休眠的影响、息屏时面板的处理方式、佩戴检测方式、存储分区大小、片内双 bank 容量是否够用、蓝牙模块分工、Gadgetbridge DFU 兼容性。这些是基于调研的设计推论，没有在 X-Watch 硬件上验证过。